SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO1 - ASSESSMENT TOOL 2

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.

(BL-1, BL-2, BL-3)

Assessment Tool Weightage:

QUESTIONS: 10 **Total Marks: 100**

Annexure A

Concept Mapping

Code of Conduct:

I K. Srinivasa Reddy (192525452)
(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. SrinivasaReddy

Interpret how this concept map helps in classifying algorithms based on their order of growth.

10. Consider the concept map: Search Algorithm → Linear Search → O(n) and Search Algorithm → Binary Search → O(log n). Construct the concept map with two parallel branches and explain how each searching approach leads to a different complexity result. Extend the map by adding Input Requirement as a node connected to Binary Search to indicate the need for sorted data. Justify why the growth rate of binary search is better than that of linear search for large inputs. Interpret how the comparative structure of the map supports algorithm selection.

RUBRICS FOR CALCULATION

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

1. Consider the concept map: Algorithm → Input Size (n) → Time Complexity → Asymptotic Notation → Efficiency. Explain the directional relationship between each node and show how an algorithm is evaluated as the input size increases. Extend the map by adding the branches Big-O, Big-Theta, and Big-Omega under Asymptotic Notation. Justify how these notations help classify the growth of running time for large values of n. Interpret how the completed map supports the measurement of algorithm efficiency in a structured way.

2. Consider the concept map: Algorithm → Best-case, Average-case, Worst-case → Time Complexity → Performance Evaluation. Map the relationships among the three cases and explain how they provide different views of algorithm behavior. Extend the map by linking each case to Asymptotic Notation so that the complexity can be expressed formally. Justify why all three cases are important in performance analysis rather than relying on only one measure. Interpret how the overall structure helps compare the practical and theoretical behavior of algorithms.

3. Consider the concept map: Divide and Conquer → Recurrence Relation → $T(n) = aT(n/b) + f(n)$ → Master Theorem. Explain how each node leads to the next in the analysis of recursive algorithms. Extend the map by adding the three cases of the Master Theorem as child nodes and show how each case depends on the comparison between f(n) and n^log_b a. Justify how this mapping simplifies the process of deriving time complexity. Interpret how the completed concept map helps students analyze divide-and-conquer algorithms more efficiently.

4. Consider the concept map: Recurrence Relation → Substitution Method → Expansion → Guess → Induction → Verification → Complexity Derivation. Trace the logical flow from a recurrence relation to its final complexity using the substitution method. Add suitable directional links to show how guessing and induction support the proof of the derived complexity. Justify the role of verification in confirming the correctness of the assumed solution. Interpret how this map provides a stepwise framework for solving recurrence relations systematically.

5. Consider the concept map: Recurrence Relation → Recursion Tree → Level Cost → Height of Tree → Total Cost → Time Complexity. Explain

how a recurrence can be represented using a recursion tree and how cost is distributed across levels. Extend the map by including branching factor as a supporting node connected to the tree structure. Justify how summing the level-wise costs leads to the final total cost. Interpret how this map makes recursive complexity analysis more visual and easier to understand.

6. Consider the concept map: Algorithm → Operation Count → Growth Function → Big-O Notation → Upper Bound. Map the conceptual links showing how counting basic operations leads to a growth function and then to an asymptotic upper bound. Extend the map by connecting Worst-case Analysis as a supporting node to Big-O Notation. Justify why Big-O is used to represent an upper limit on running time rather than an exact value. Interpret how the completed structure supports reliable performance estimation for large inputs.

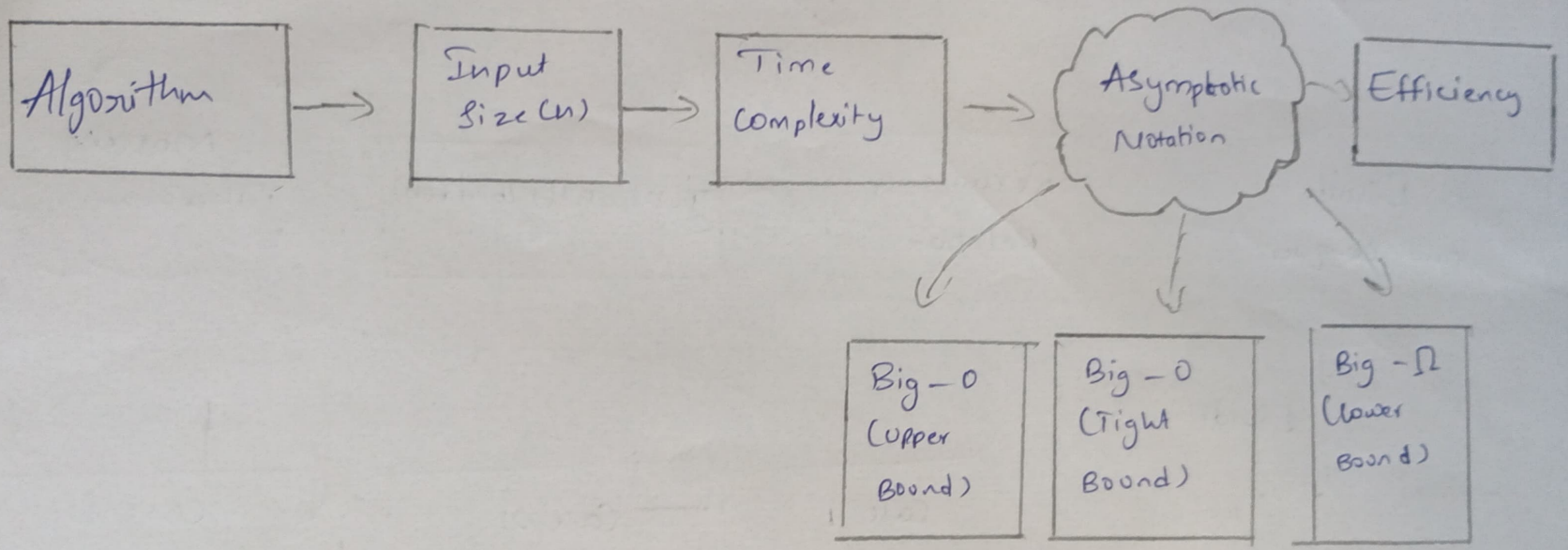
7. Consider the concept map: Algorithm → Growth Function → Big-Omega Notation → Lower Bound → Best-case Analysis. Explain the relationships among the nodes and show how lower-bound analysis captures the minimum behavior of an algorithm. Extend the map by linking Performance Guarantee and Input Characteristics as additional supporting nodes. Justify why Big-Omega is useful when discussing guaranteed minimum efficiency. Interpret how the concept map gives a balanced understanding of algorithm behavior from the lower-bound perspective.

8. Consider the concept map: Algorithm → Growth Function → Big-Theta Notation → Tight Bound → Exact Order of Growth. Explain how the concept of tight bound emerges when upper and lower bounds meet. Extend the map by adding links from Big-O and Big-Omega toward Big-Theta to show their relationship. Justify why Big-Theta is often preferred when the exact asymptotic growth is known. Interpret how the map helps distinguish between approximate and precise complexity descriptions.

9. Consider the concept map: Algorithm → Input Size → Constant Time, Linear Time, Logarithmic Time, Quadratic Time → Efficiency Classification. Explain how changes in input size lead to different growth patterns and how these patterns are classified. Extend the map by linking Asymptotic Notation to each complexity class. Justify why logarithmic and linear algorithms are generally more scalable than quadratic algorithms.

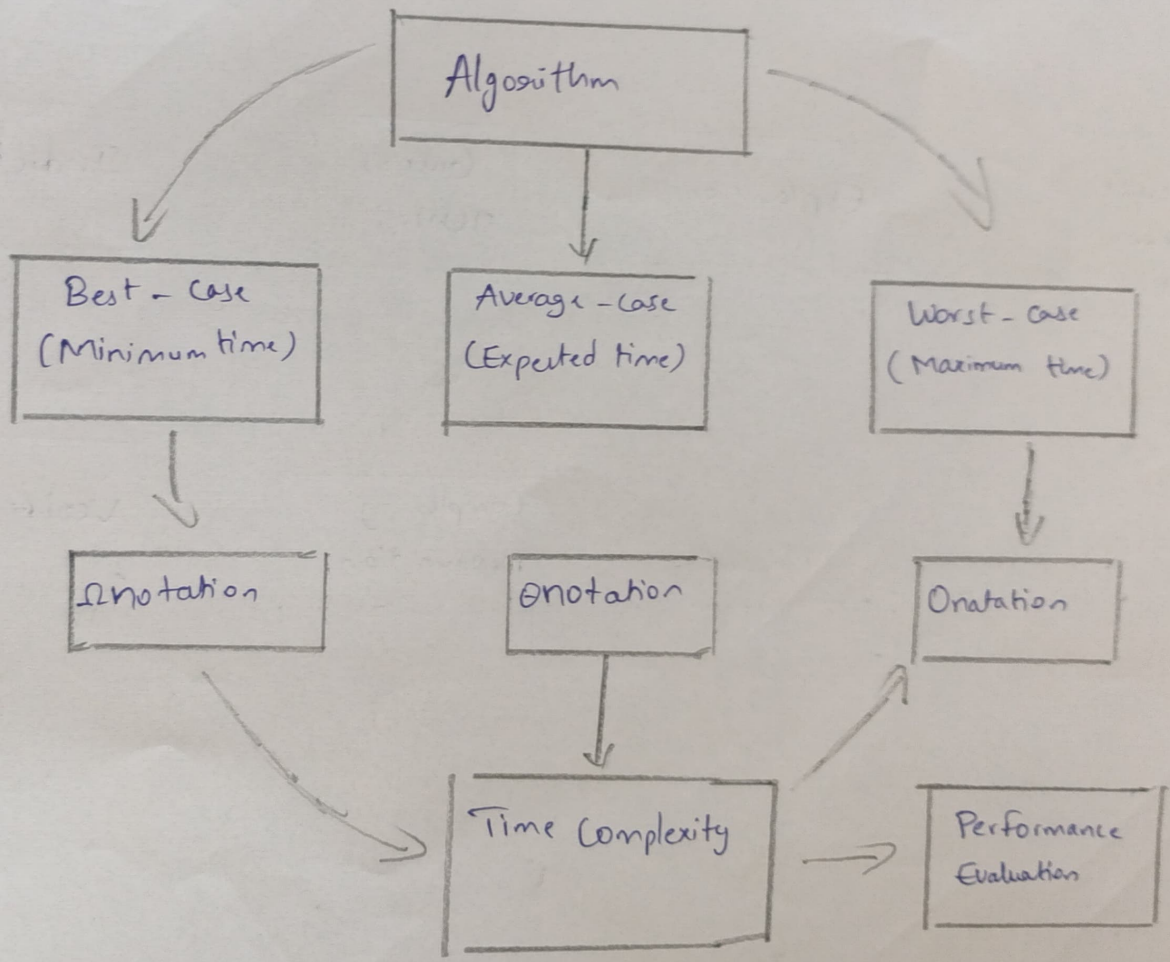
(1)

Algorithm → Input Size → Time Complexity → Asymptotic Notation → Efficiency



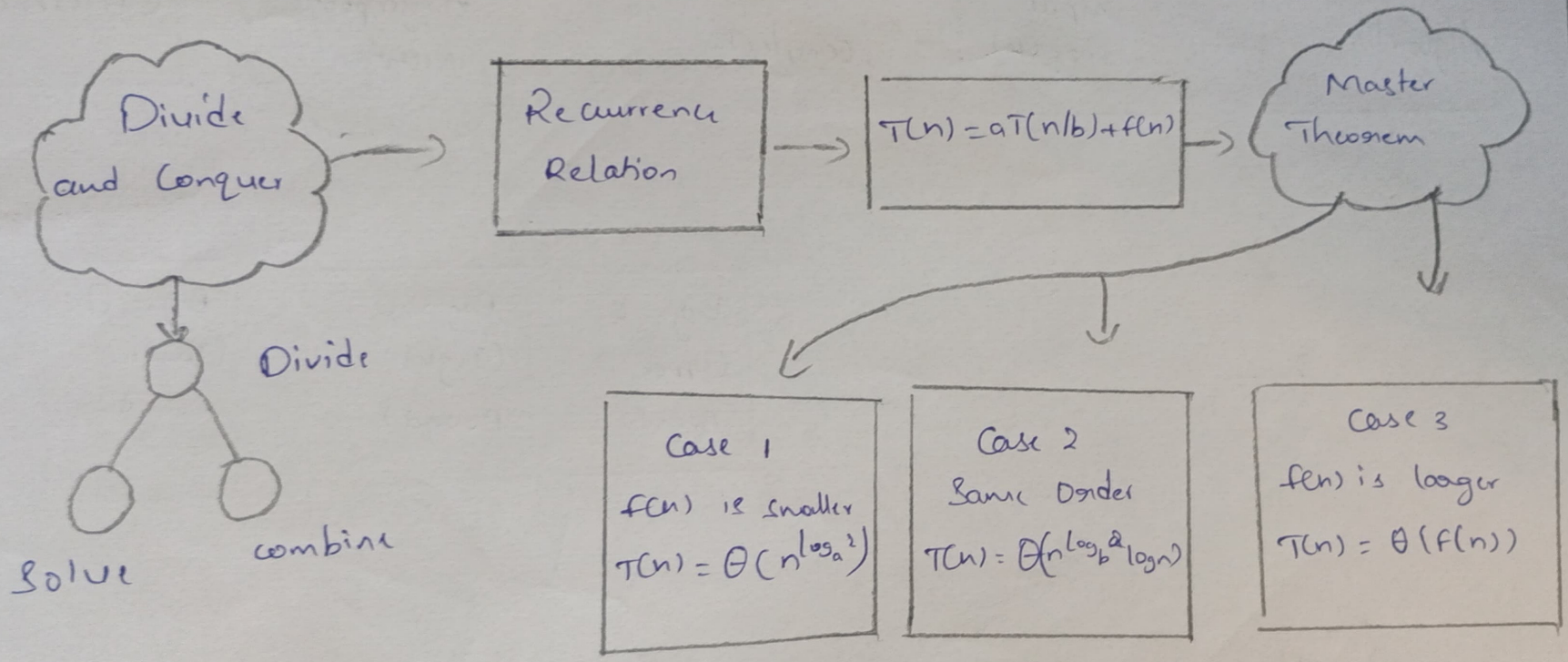
Performance Evaluation.

② Best-case, Average-case, Worst case → Time complexity → Performance Evaluation



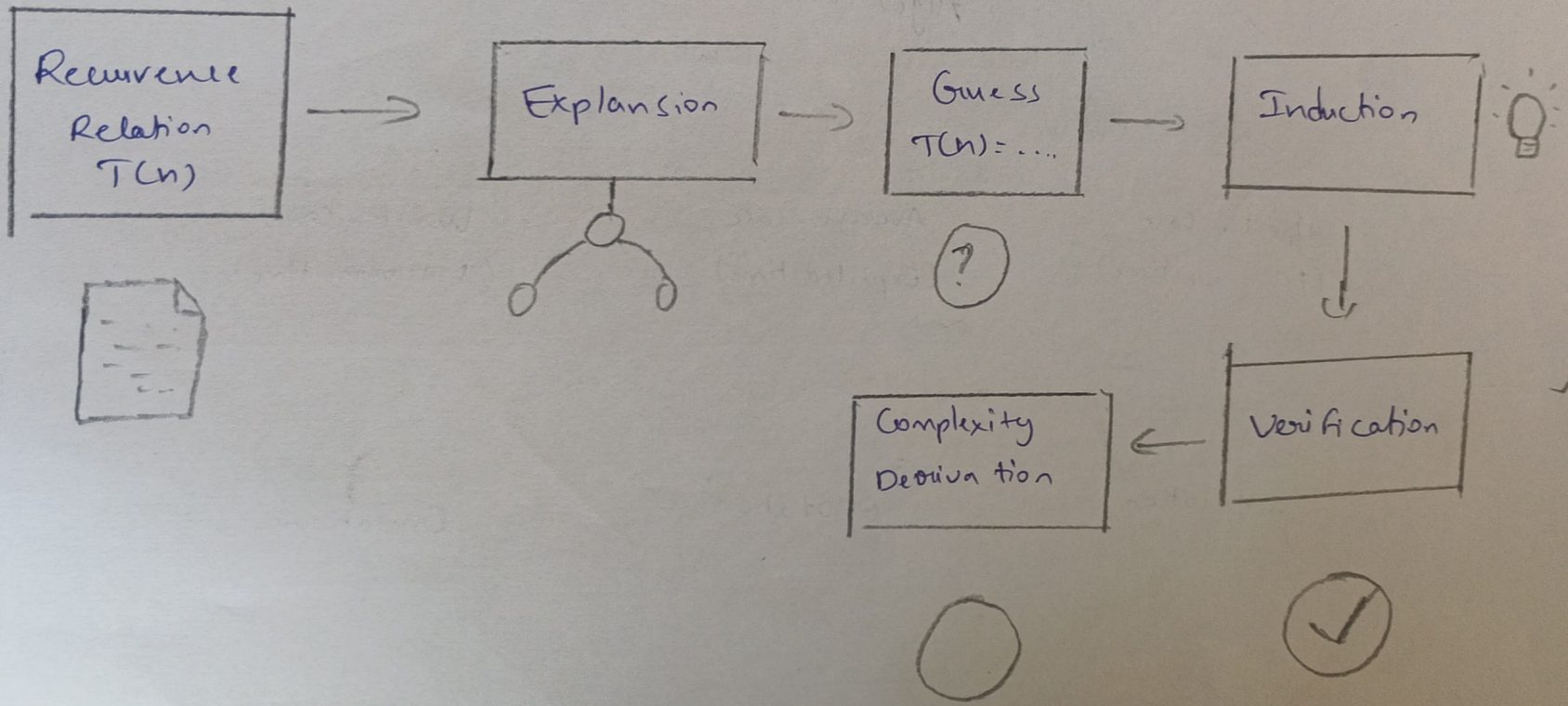
3

Divide and Conquer ⟶ Recurrence Relation ⟶ Master Theorem

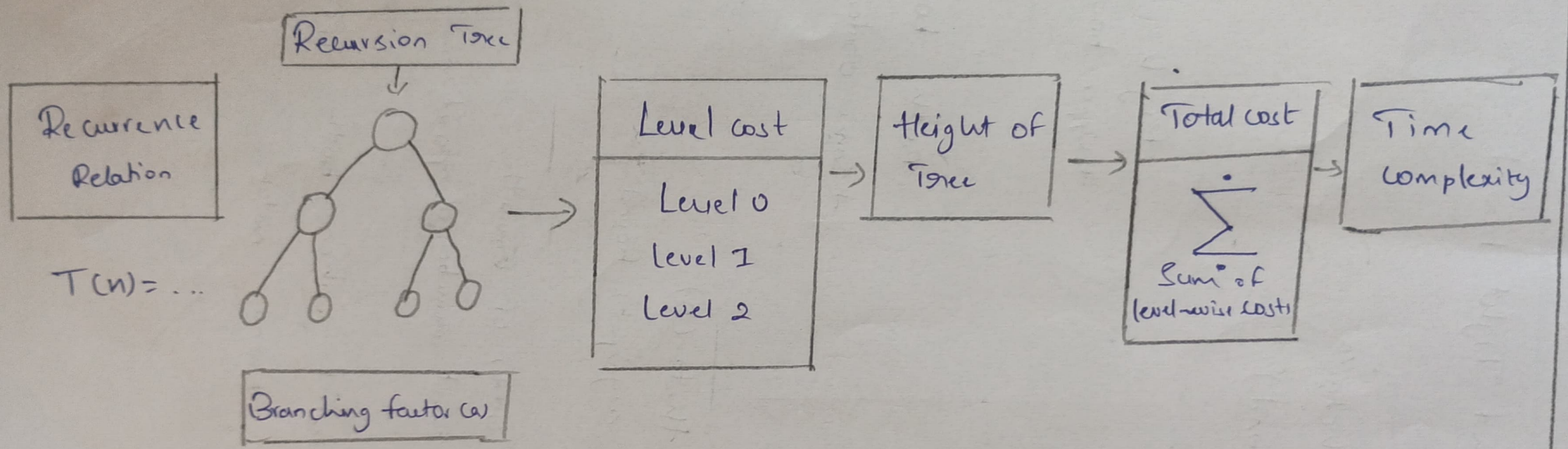


4

Recurrence Relation → Substitution Method

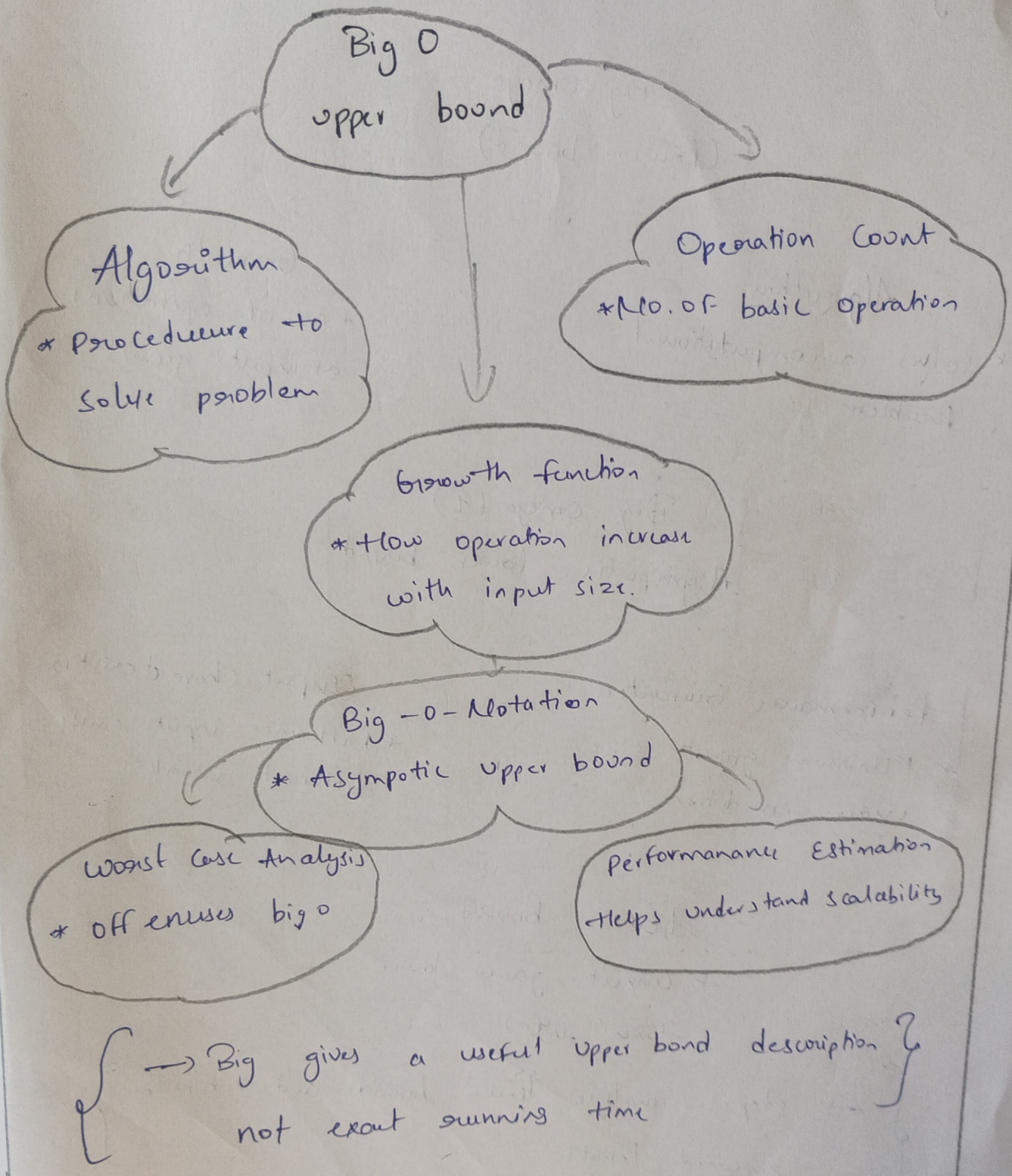


(3) Recurrence Relation → Recursion Tree → Level cost → Total cost → Time complexity



* For $T(n) = 2T(n/2) + n$: level 0 costs n; level 1 has two nodes costing $n/2$ each, total n; level 2 has four nodes costing $n/4$ each, total n. This continues until subproblem size becomes 1.
* The branching factor is 2 and the tree height is approximately $\log n$. There are $\Theta(\log n)$ levels, each costing $\Theta(n)$. So $T(n) = \Theta(n \log n)$.

(6) # Big O Notation & upper bound



→ Big gives a useful upper bond description
not exact running time

7 Big- Omega Notation & Lower bound

Big omega
(Lower bound)

Algorithm
* Solves a computational problem

Growth
* NO. of operations as input grows

Big omega (Ω)
Asymptotic lower bound

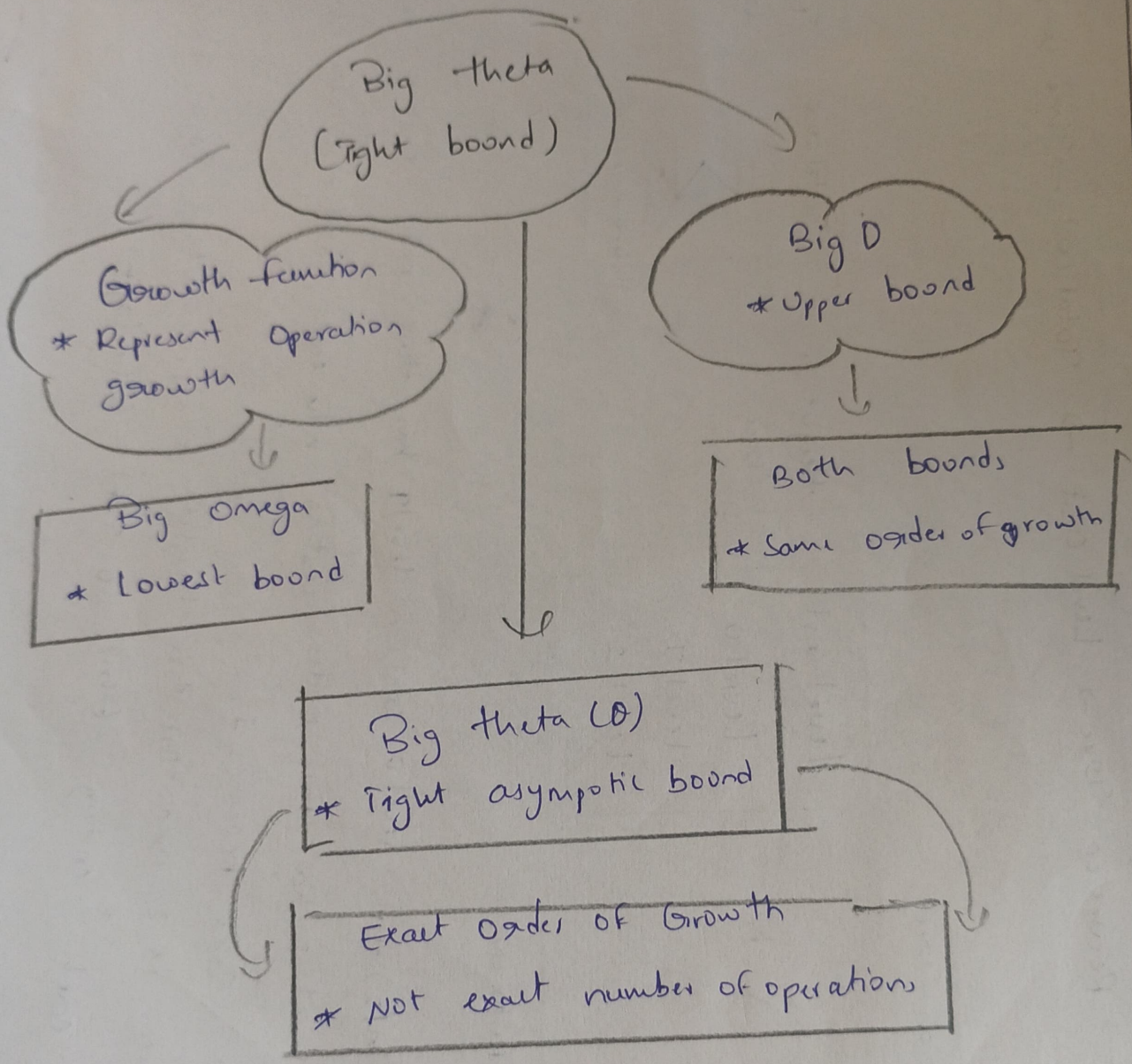
Performance Guarantee
* Minimum growth size under conditions

Input Characteristics
* Different inputs cause different work

Lower bound
* cannot grow slower than Ω(n)

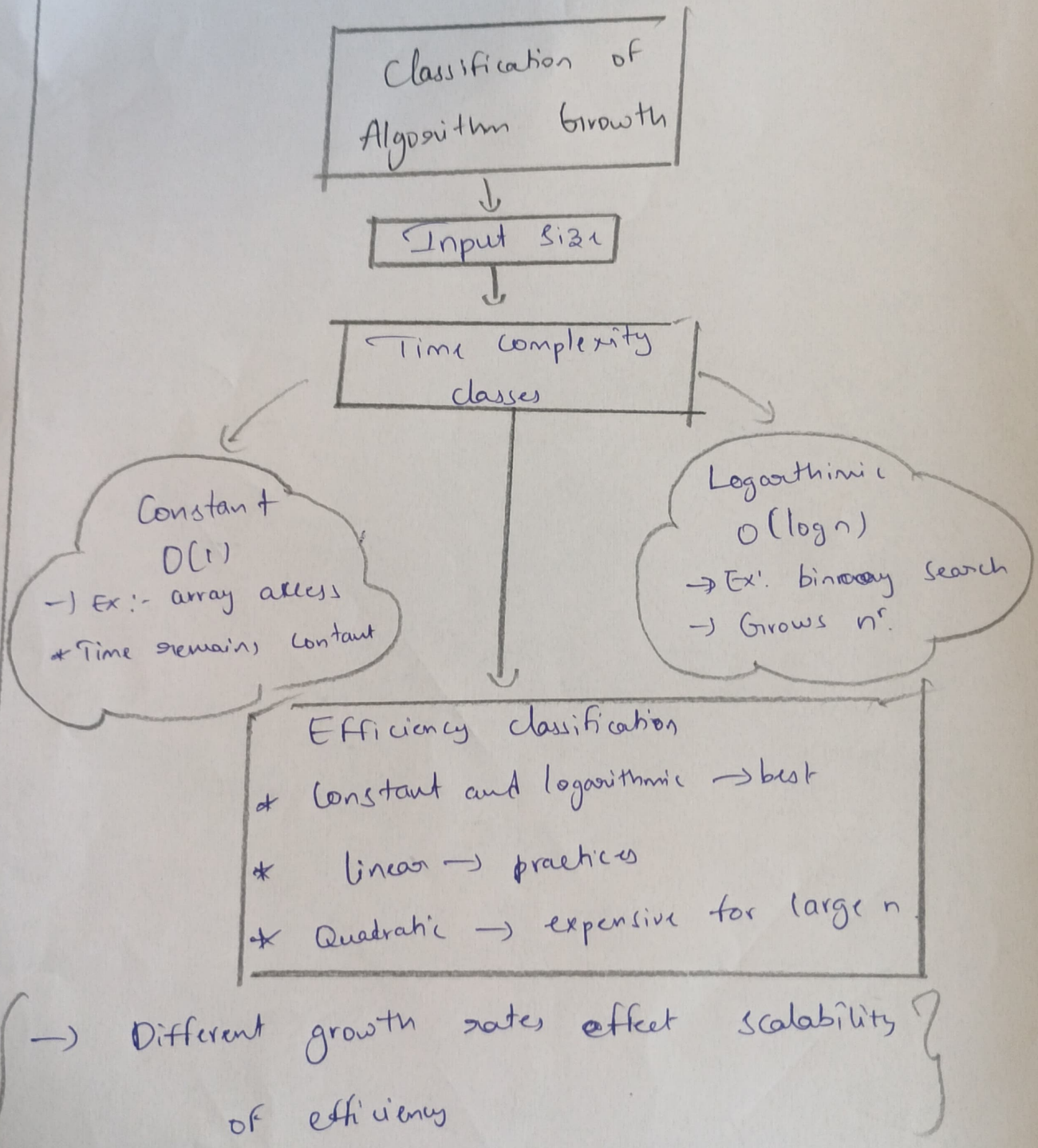
→ steps analyse lower bounds and minimum required length

⑧ Big Θ - Notation and Tight bound.



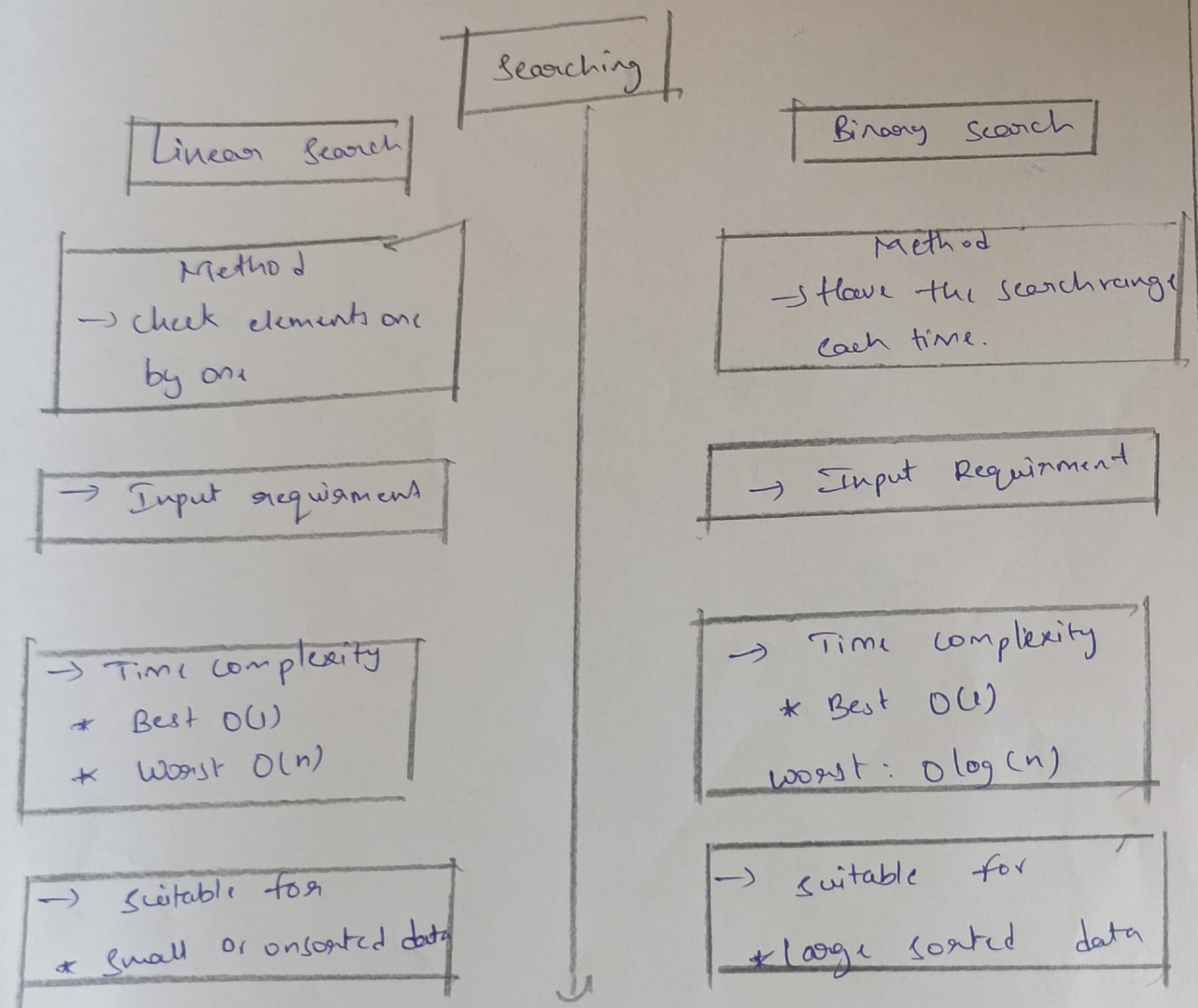
→ Big O is useful when both upper and Lower bounds match

(9) Types of time complexity



→ Different growth rates affect scalability of efficiency

(10) Linear Search Vs Binary Search



Eg: (1024 elements).

* linear Search : upto 1024
* Binary Search : about 11 comparisions

→ Binary search is faster for large sorted datasets, while linear search is simplified